

Dokumentácia k úlohe č. 3

Podúloha 2

Cieľ zadania:

Vyvinúť a navrhnuť neurónovú sieť MLP (Multi Layer Perceptron), ktorá bude schopná analyzovať a učiť sa lineárne neoddeliteľné problémy, ako napríklad XOR. Neurónová sieť musí byť implementovaná od začiatku a musí pozostávať z manuálneho algoritmu spätného šírenia, smerov dopredu a dozadu. Okrem toho sa má implementovať technika momentumu spolu s SGD (Stochastic Gradient Descent).

Koncepcia práce pre backpropagation

Aby sme pochopili, ako funguje spätné šírenie, je potrebné porozumieť smeru výpočtu dopredu. Smer výpočtu dopredu je v podstate funkcia, ktorá prijíma vstup, aplikuje naň vnútorné výpočty a výsledkom je výstup.

V kóde sa funkcia `forward()` používa na výpočet výstupu vrstvy. Lineárna vrstva sa skladá z matice váh W a vektora predpätí B . Keď sa vstup, napr. X , odovzdá do funkcie, výstup sa vypočíta pomocou nasledujúceho vzorca:

$$y = W * X + B$$

Potom je odoslaný z funkcie ako výstup. V aktívnej vrstve sa na transformáciu vstupu používa iná funkcia. Môže to byť ReLu, tanh, Sigmoid atď. A spätné šírenie je mechanizmus, vďaka ktorému sa model dokáže učiť.

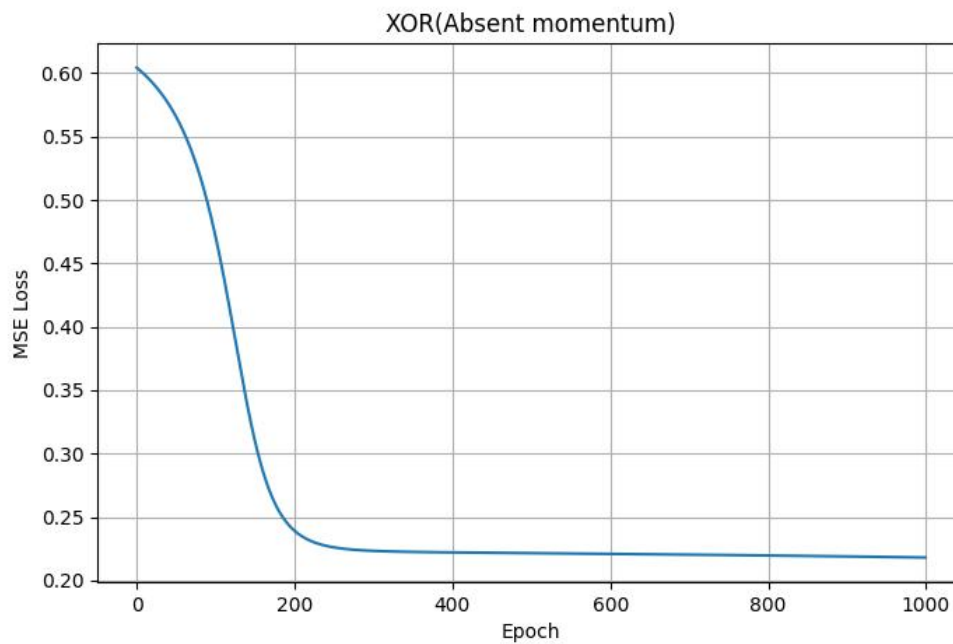
Aby sa model mohol učiť, musia sa zmeniť jeho váhy. A aby sa menili správnym smerom, derivácia stratovej funkcie sa počíta vo vzťahu k ostatným vrstvám, konkrétne funkcii, v neurónovej sieti. Počítajú sa, aby sa zistilo, či má funkcia sklon s hodnotou odlišnou od nuly, alebo nie. V prípade, že je odlišná od nuly, funkcia smeruje k zníženiu svojej chyby. A aby sa „pohla“, upravuje svoje parametre, t. j. mení váhy.

Zmenia sa výpočtom derivácie každej stratovej funkcie vzhľadom na funkciu každej vrstvy a následným prenesením všetkých potrebných hodnôt do týchto derivácií. Neskôr sa pre aktuálne váhy každej vrstvy vypočítaná derivácia odpočíta od váh jej vrstvy. Aby však model nekolísal, tieto vypočítané derivácie sa vynásobia rýchlosťou učenia, aby model „nepreskočil“ minimum.

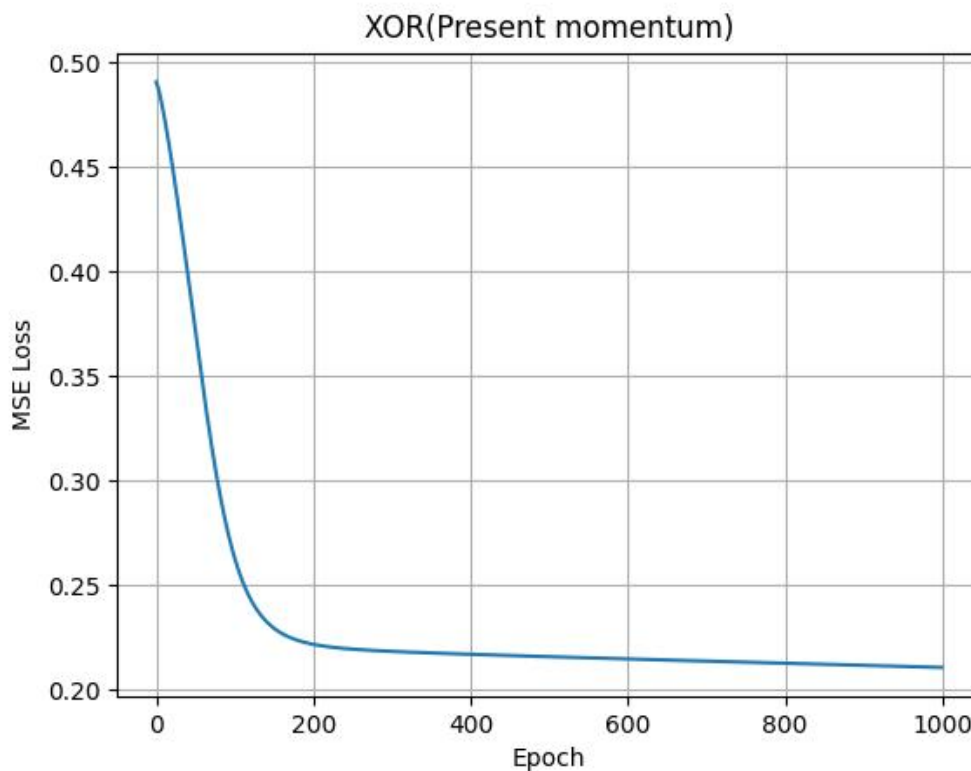
A na výpočet derivácie každej vrstvy je potrebná derivácia predchádzajúcej vrstvy. Takže po tom, ako posledná vrstva vypočíta svoju deriváciu a odovzdá hodnoty do funkcie derivácie, prenesie výsledok do predchádzajúcej vrstvy. Predchádzajúca vrstva vezme deriváciu nasledujúcej vrstvy a s jej pomocou vypočíta svoju vlastnú, aktualizuje svoje váhy a odovzdá vypočítanú deriváciu predchádzajúcej vrstve. Tento mechanizmus sa nazýva spätná propagácia, pretože propagujeme späť, súčasne aktualizujeme váhy jednotlivých vrstiev a nakoniec aktualizujeme váhy celého modelu.

Experimentálna časť

Poznámka: experimenty tu používajú počet epoch 1000, pretože pri menšom počte nie sú výsledky tak zrejmé, ako by mali byť. V časti hodnotenia modelu s počtom epoch 500 a rýchlosťou učenia 0,021 je strata modelu približne 30 %. Sieť pre tento konkrétny prípad mala 1 skrytú vrstvu (4 neurónové jednotky) a 2 aktívne funkcie, sigmoid a tanh, pričom MSE bola funkciou straty.



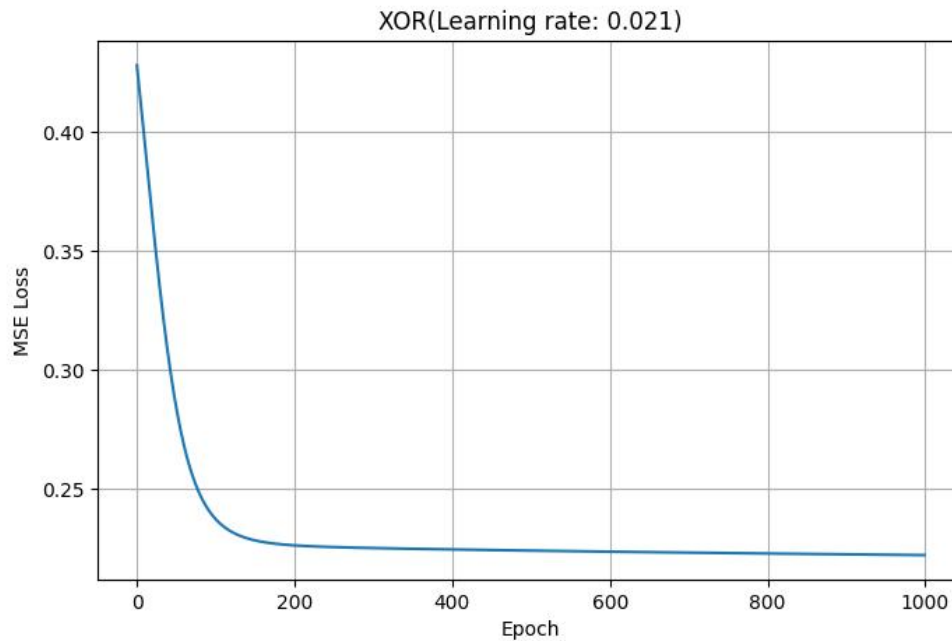
Obrázok 1. MLP s obyčajným SGD



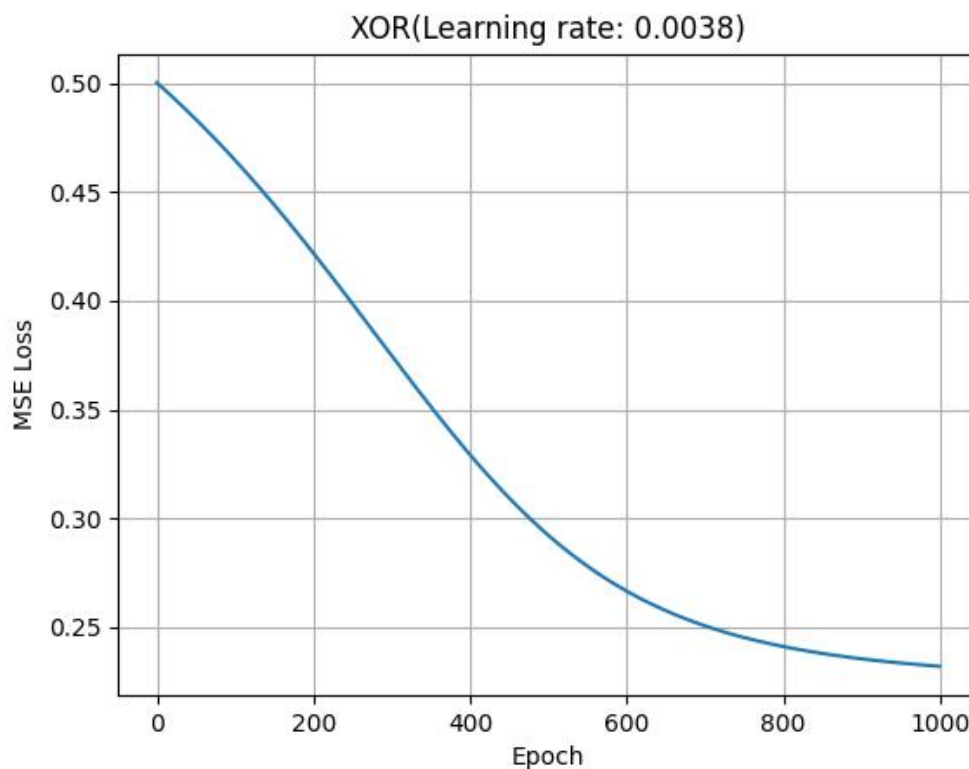
Obrázok 2. MLP s SGD a využitím momentumu

Ako je vidieť z obrázku 1 aj obrázku 2, s nárastom počtu spracovaných vzoriek sa zodpovedajúcim spôsobom zvyšuje celková strata modelu. Okrem toho oba grafy ukazujú rýchly pokles chyby modelu, až kým nedosiahne hodnotu približne 0,23. Potom modely prestanú učiť sa tak rýchlo a spomalia svoje krivky učenia. Na rozdiel od modelu bez momentumu, kde výkon modelu stále rastie, t. j. model sa stále učí, model s momentumom aplikovanou v bode 150 až 180 spracovaných iterácií prestáva učiť úplne a vykazuje rovnaký výsledok ako model bez momentumu, 20 až 50 iterácií skôr.

Hoci to v súčasnom príklade ešte nie je veľmi zrejmé, s malou funkciou na učenie je aj tu rozdiel nepatrný, ale viditeľný.



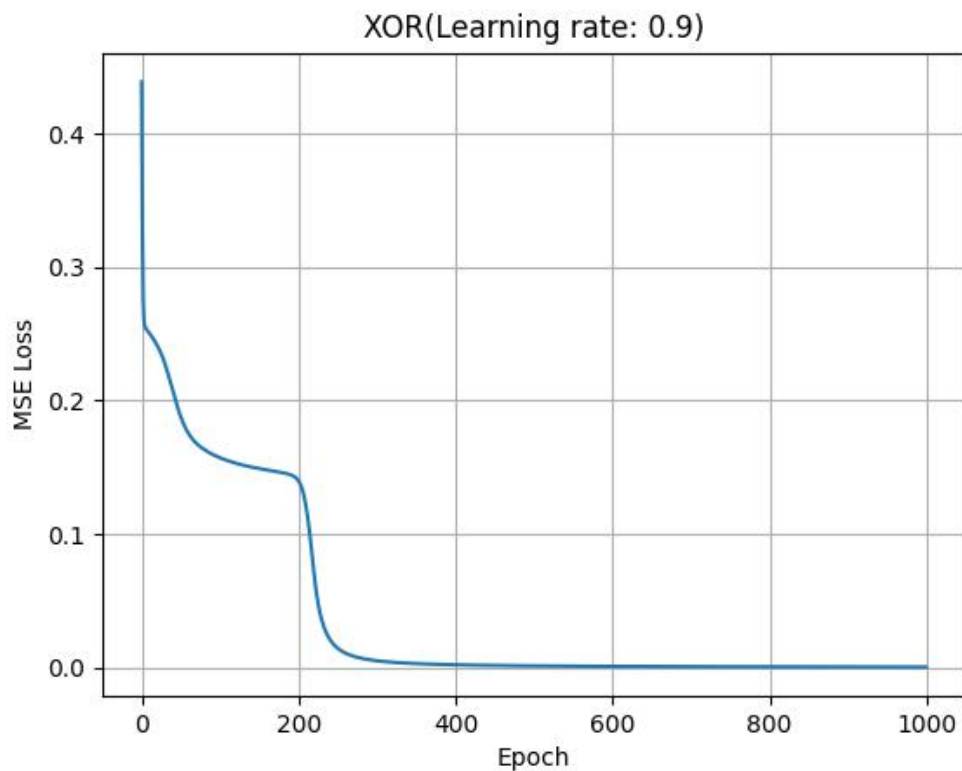
Obrázok 3. Model s optimalizovanou rýchlosťou učenia



Obrázok 4. MLP trénovaný s použitím relatívne malej rýchlosti učenia

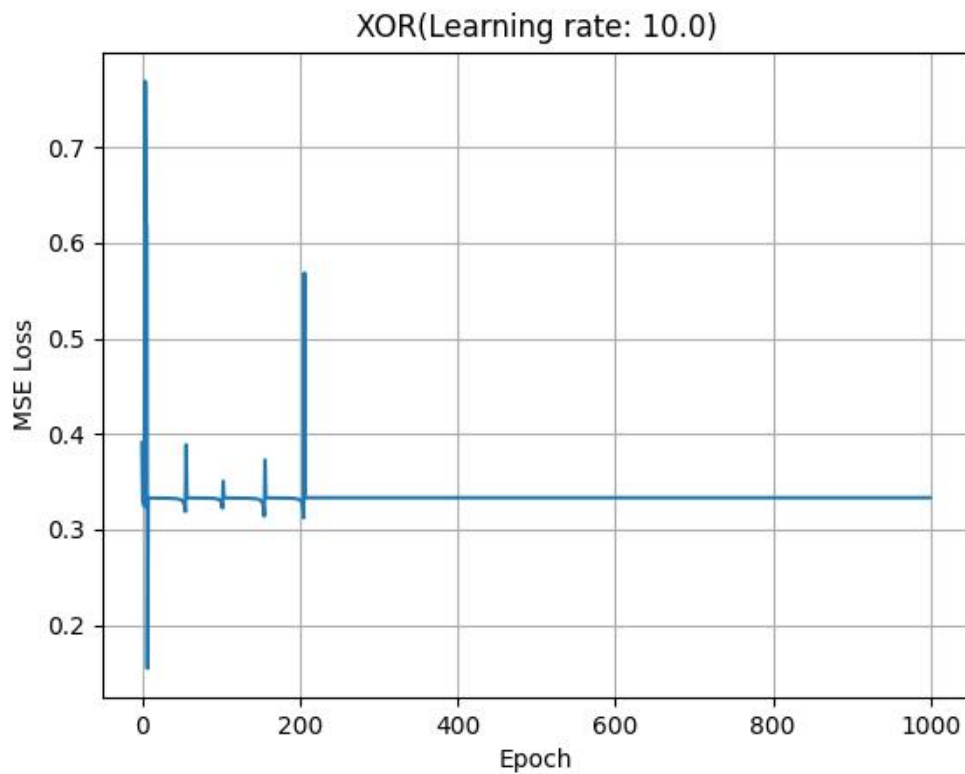
Rozdiel je veľmi viditeľný a je ľahké rozlíšiť model s optimalizovanou rýchlosťou učenia od modelu s príliš malou rýchlosťou učenia. V podstate kvôli tomu, že model s menšou rýchlosťou učenia mení svoje váhy vo veľmi malých častiach, chyba zostáva vysoká počas značnej časti tréningového času. Ale neskôr, keď sa model naučí všetky potrebné vzory, je takmer rovnako

výkonný ako model s optimalizovaným MLP. Nevýhodou príliš nízkej rýchlosti učenia je však čas, ktorý je potrebný na dosiahnutie rovnakého percenta chyby ako v prípade modelu s vyššou rýchlosťou učenia.



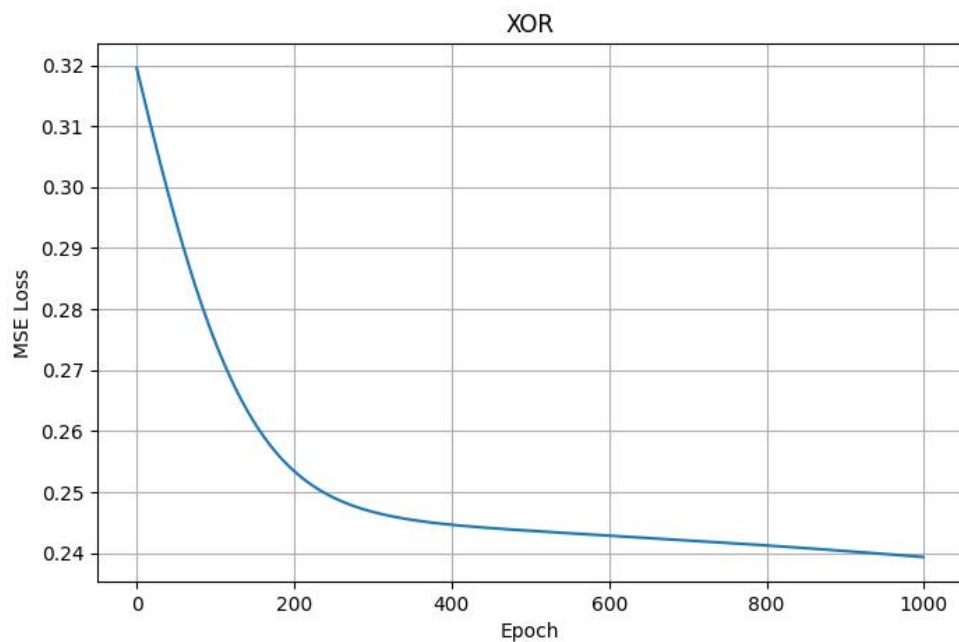
Obrázok 5. Model s relatívne miernou rýchlosťou učenia

V konkrétnom prípade XOR, ktorý je jednoduchou funkciou, môže súčasný model fungovať dobre aj s rýchlosťou učenia 0,9, čo sa považuje za obrovské v porovnaní s väčšími a zložitejšími modelmi. Ale čo ak sa pokúsime rýchlosť učenia ešte zvýšiť?

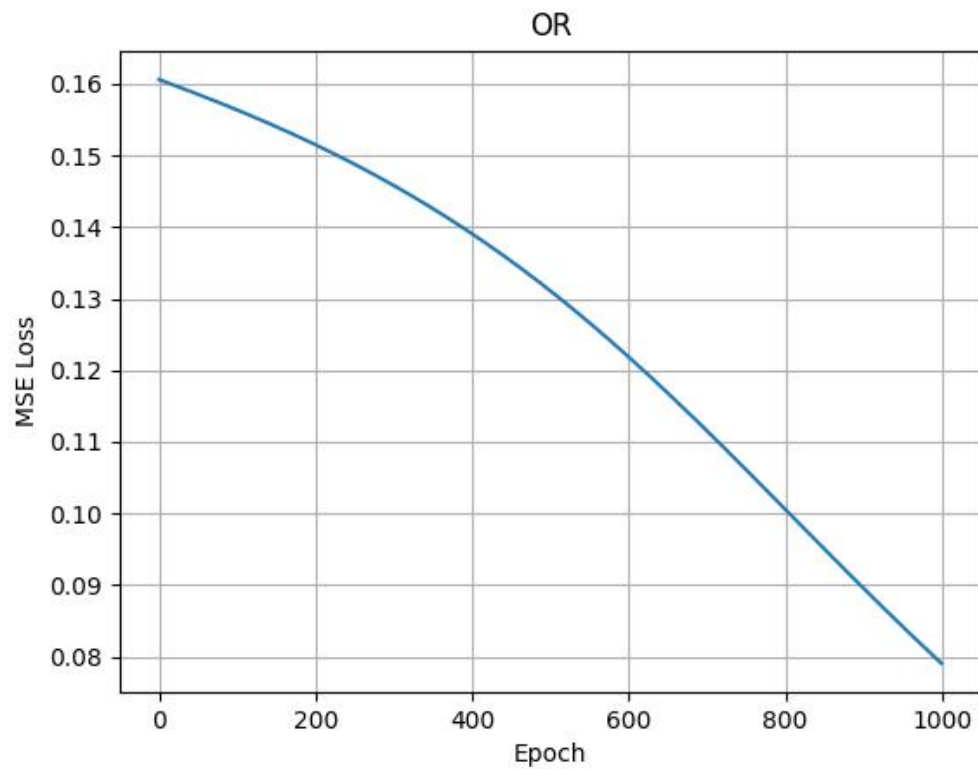


Obrázok 6. Model s rýchlosťou učenia 10,0

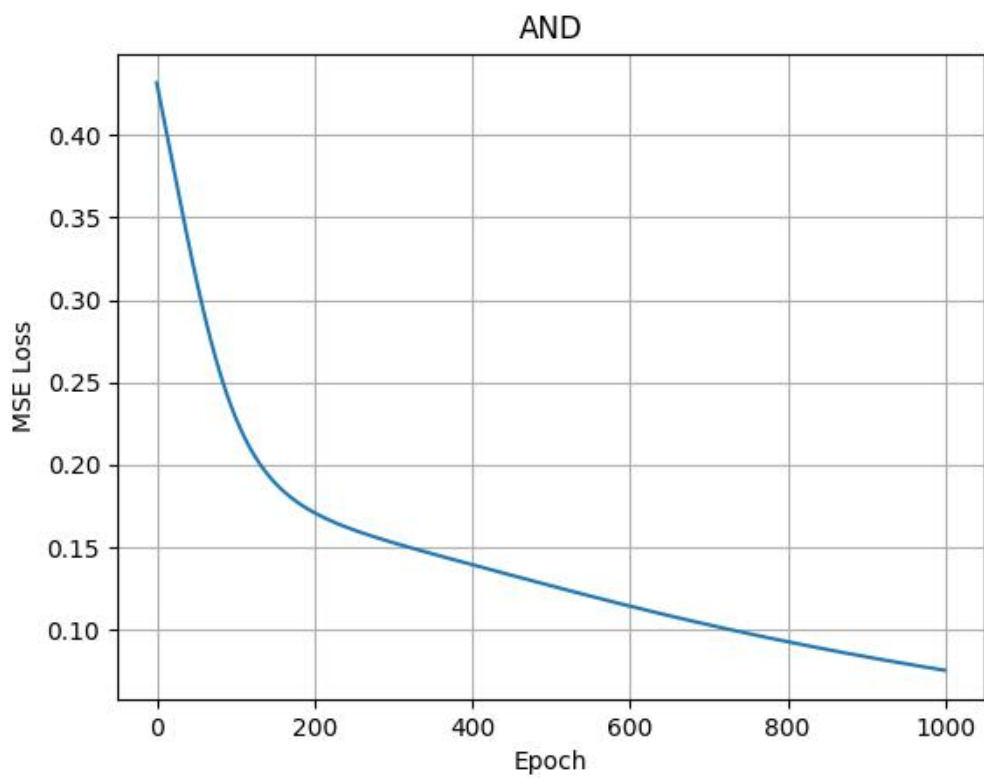
V tomto prípade sa však model nič nenaučil, pretože len prudko osciluje. V skutočnosti je horná známa prahová hodnota pre tento model pravdepodobne 10,0, ale pre akýkoľvek väčší model s väčším počtom dátových bodov, ktoré sa majú naučiť, je táto prahová hodnota obrovská a aj hodnota 1,0 by pre ne bola veľkou rýchlosťou učenia. Pretože dátový súbor obsahuje len 4 body a povrch straty je veľmi hladký, rýchlosti učenia až do $\approx 2,0$ zostávajú stabilné – čo je jav, ktorý sa pri väčších dátových súboroch vyskytuje len zriedka.



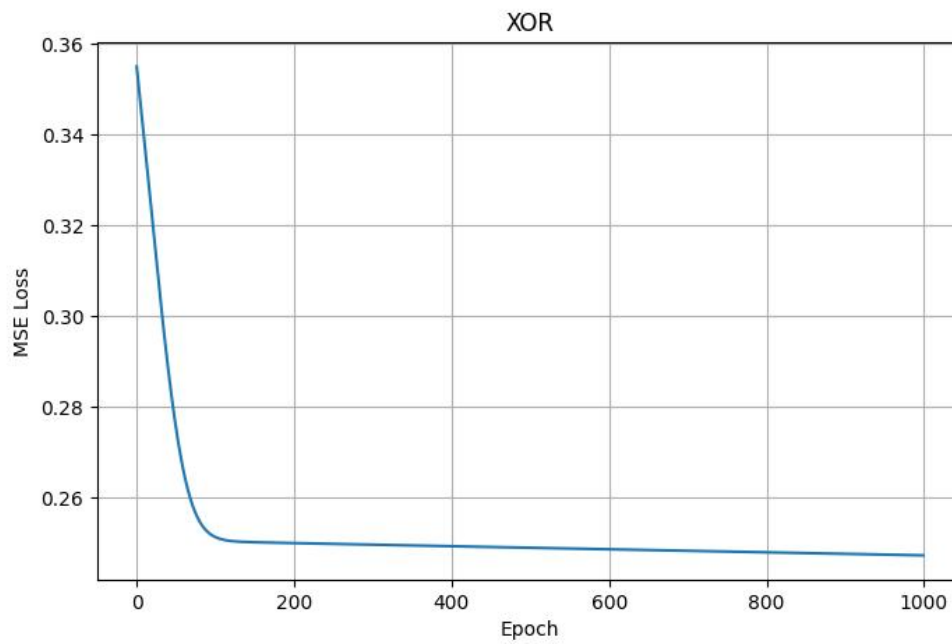
Obrázok 7. MLP s 1 skrytou vrstvou a aplikáciou momentumu



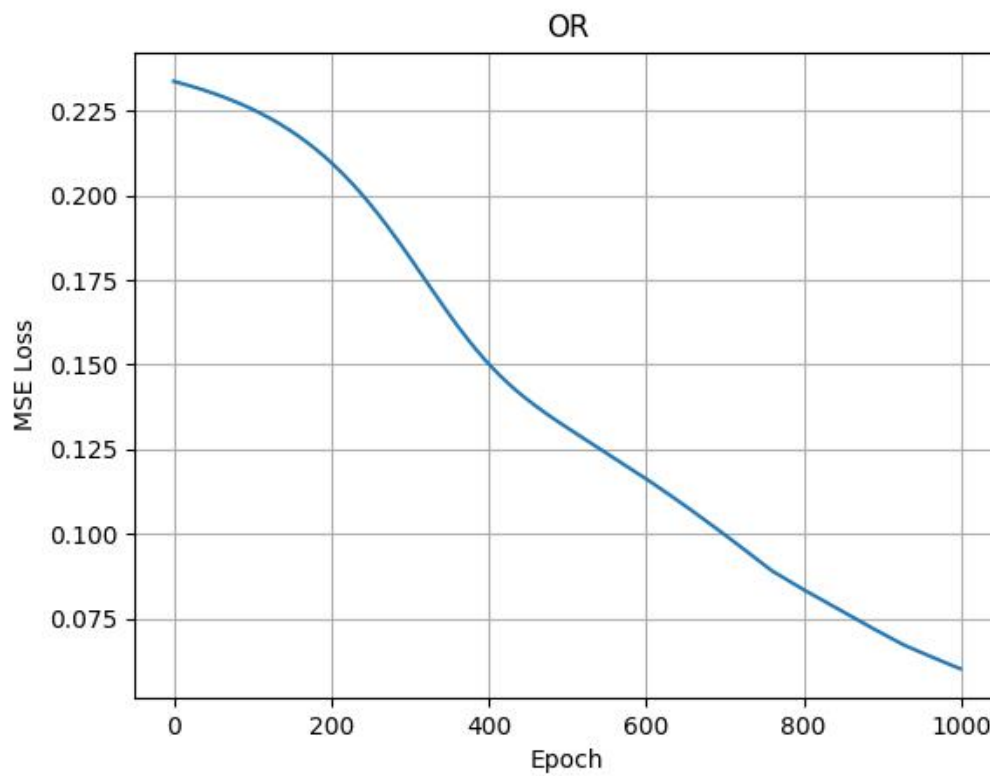
Obrázok 8. MLP s 1 skrytou vrstvou a aplikáciou momentuma



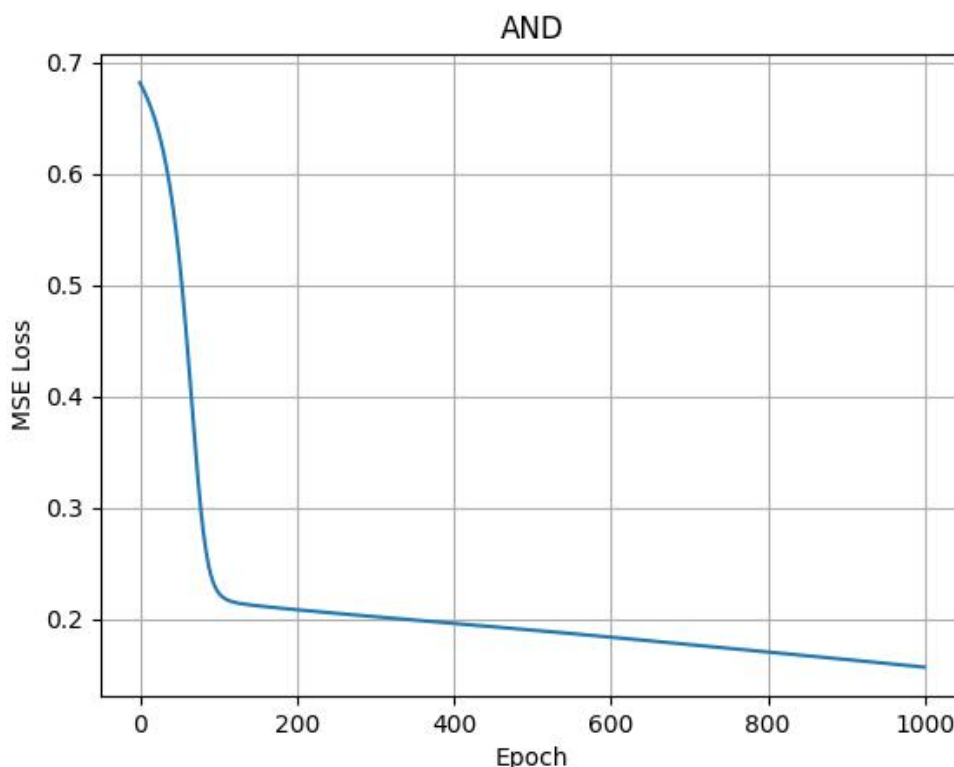
Obrázok 9. MLP s 1 skrytou vrstvou a aplikáciou momentumu



Obrázok 10. MLP s 2 skrytými vrstvami a aplikáciou momentuma



Obrázok 11. MLP s 2 skrytými vrstvami a aplikáciou momentumu



Obrázok 12. MLP s 2 skrytými vrstvami a aplikáciou momentumu

Ak by sme porovnali obrázky 7 až 9 a 10 až 12, mohli by sme vyvodiť záver, že vo väčšine prípadov môže zvýšený počet skrytých vrstiev zlepšiť výkon modelu, pretože sa naučí viac. Avšak nie vždy pomáha zvýšený počet vrstiev pri zlepšení straty modelu. Prípad OR ukázal, že model sa neučí rýchlejšie, dosahuje rovnaké výsledky ako model s 1 vrstvou. V ostatných prípadoch však MLP znižuje svoje hodnotenie chyby tým, že si za rovnaké časové obdobie zapamätá viac ako jeho konkurent.

Záver

Ako záver musím povedať, že výkon modelu závisí vo veľkej miere od počtu epoch, rýchlosti učenia a počtu skrytých vrstiev. S rastúcim počtom skrytých vrstiev sa model naučí viac za rovnaký čas a dokáže sa naučiť zložitejšie vzory ako modely s menším počtom vrstiev. Navyše, vysoká rýchlosť učenia môže narušiť proces štúdia údajov modelom a môže ho nakoniec zastaviť. Na druhej strane, príliš nízka rýchlosť učenia môže spomaliť model a jeho tréning bude trvať dlhšie.

Modely s momentumom sa učia rýchlejšie ako modely bez momentumu, najmä ak je učenie hyperplochy veľmi zložitá. Pridaním momentumu ($\beta = 0,9$) sa dramaticky zrýchli konvergencia na XOR v porovnaní s bežným SGD.